

Arquitectura de Software Microservicios - REST

Prof. Dr. Daniel Riesco

Web Services: Sistemas Distribuidos

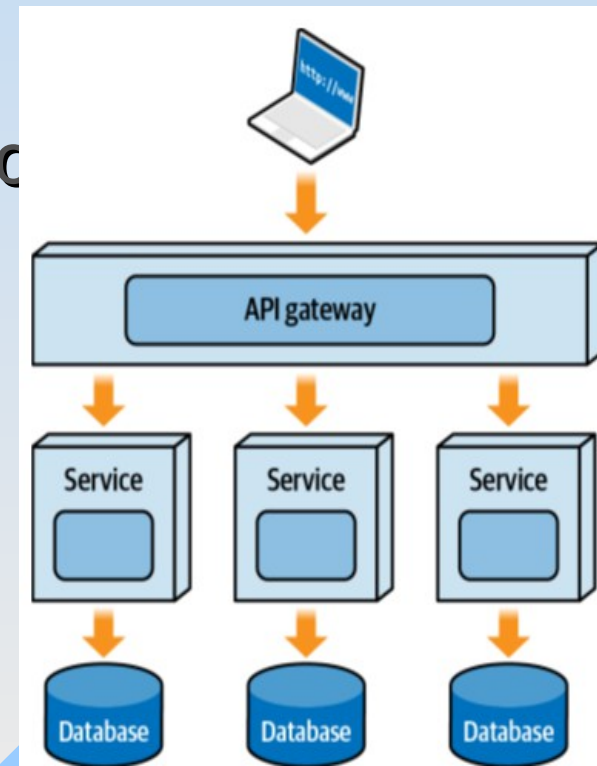
- Es uno de los paradigmas dominantes para el desarrollo de sistemas distribuidos basado en internet.
- Habilita la integración B2B. Una organización puede usar CORBA y otra .NET, y no genera problemas.
- Habilita SOA (Arquitectura Orientada a Servicios).
- Puede existir internamente a una organización o externamente en la nube.

Dos Aproximaciones

- Servicios Web basados en SOAP (WS-*)
- MicroServicios (estilo REST - REpresentational State Transfer)

Arq. Microservicios

- Es un ecosistema compuesto por servicios de propósito único implementados por separado a los que se accede a través de una puerta de enlace API.
- Cada servicio accede a sus propios datos o realiza solicitudes a otros servicios para acceder a datos que el servicio no posee.
- No requiere una BD para cada servicio.
- Puede ser una única.
- El servicio responsable de sus datos.
- API oculta la ubicación de servicios.



Que es un microservicio

- Un microservicio se define como una unidad de software de propósito único, implementada por separado, que hace una cosa muy, muy bien.
- De hecho, aquí es donde el término “microservicios” recibe su nombre: no por el tamaño físico del servicio (como el número de clases), sino por lo que hace.

REST: Beneficios

- Escalabilidad
- Simplicidad de la interface
- Modificabilidad de las componentes (mínima cohesión)
- Portabilidad: independiente de la plataforma y el lenguaje
- Confiabilidad: la restricción sin estado permite una recuperación más fácil de un sistema después de una falla.
- Tolerante a falla: una falla de un componente no afecta toda la estabilidad del sistema.
- Facilidad de interconexión de componentes.
- Tecnologías Heterogéneas

REST: Desafíos

- **Complejidad Operacional:** Gestionar la infraestructura, monitoreo y depuración debido al número de servicios involucrados.
- **Gestión de Datos Distribuidos:** Requiere una estrategia para la consistencia y sincronización de datos entre servicios.
- **Seguridad y Comunicación Interservicios:** Implementar y asegurar las comunicaciones entre servicios puede ser complejo y requiere mecanismos de autenticación y autorización robustos.
- **Orquestación y Coreografía:** La coordinación de procesos distribuidos es un desafío, especialmente con transacciones que abarcan múltiples servicios.

Arq. Microservicios: Cuando usarlo

- ¿Es factible dividir la funcionalidad de su aplicación en docenas o cientos de piezas de funcionalidad separadas y distintas que sean independientes entre sí?
- Aplicaciones que requieren altos niveles de agilidad
Despliegues rápidos, en lugar de despliegues a gran escala durante un fin de semana.
- Aplicaciones con alta tolerancia a fallas y necesidades de alta escalabilidad.
- Agregar funcionalidad es crear un servicio.

Microservicios: Cuando NO usarlo

- Cuando la funcionalidad implementada por separado debe vincularse con flujos de trabajo complejos y mucha comunicación u orquestación entre servicios.
- Datos están estrechamente acoplados y son de naturaleza monolítica.
- Latencia: latencia de red, latencia de seguridad y latencia de datos. Seguridad verificar cada invocación al servicio. La latencia de datos es la cantidad de tiempo que tardan otros servicios en consultar, en su nombre, datos que no son de su propiedad.

Principios Arquitectónicos de REST

- La web tiene recursos que son direccionables.

Cada recurso tiene un URI.

- La web tiene una interfaz uniforme y restringida.

HTTP tiene un número pequeño de métodos que se usan para manipular recursos.

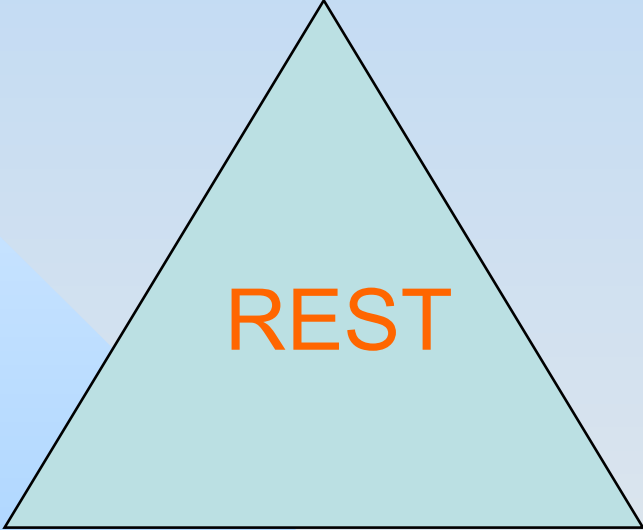
- La web está orientada a la representación, proporcionando diversos formatos.
- La web puede usarse para comunicación sin estado, proporcionando escalabilidad

Principales conceptos

Sustantivos (Recursos)

Sin restricciones

Ej. <http://pirulo.com.ar/empleado/12345>



REST

**Verbos
Restringidos**
Ej. GET

**Representacion
Restringido**
Ej. XML, JSON

Recursos

La abstracción clave de la información en REST es un recurso.

Un recurso es un mapa conceptual a un conjunto de entidades:

Cualquier información que puede ser nombrada puede ser un recurso: un documento o imagen, un servicio temporal (tiempo en estos momentos en CABA), una colección de recursos, un objeto no virtual (una persona), etc.

Representado con un identificador global (URI-HTTP)

<http://www.aerolineas.com.ar/aeronave/747>

Verbos

La forma de preguntar por la información que se busca

Solicitud GET transfiere datos del servicio al cliente en alguna representación

GET <http://localhost/libros>

Recupera todos los libros

GET <http://localhost/libros/ISBN-12345>

Recupera el libro identificado por ISBN-12345

GET <http://localhost/libros/ISBN-12345/autores>

Recupera los autores del libro identificado con el ISBN-12345

HTTP POST crea un recurso

POST [http://localhost/books?titulo="ABC"&ISBN="ISBN-12345"](http://localhost/books?titulo=ABC&ISBN=ISBN-12345) ...

Representacion

XML

```
<CURSO>  
  <ID>AyDII</ID>  
  <NOMBRE>Análisis y Diseño II</NOMBRE>  
</CURSO>
```

JSON

```
{ "curso": {  
  "id": "AyDII",  
  "nombre": "Análisis y Diseño II" }  
}
```

REST: Obtener imágenes de perros

- Obtener una imagen random

`https://dog.ceo/api/breeds/image/random`

- Obtener una imagen de un perro particular

`https://dog.ceo/api/breed/{breed}/images/random`

`https://dog.ceo/api/breed/labrador/images/random`

{

`"message": "https://images.dog.ceo/breeds/labrador/n02099712_4845.jpg",`

`"status": "success"`

}

REST: Estilo Arquitectónico

- REST es un estilo de arquitectura de sistemas en red.
- Ignora detalles de implementación
- Se focaliza en sus componentes, sus interacciones y su interpretación de los elementos de datos
- Recursos: URI
- Bajo el modelo Arquitectónico WWW
- Estandars de protocolos Web
- Basado en Web Services



Principio: Direccionalidad

- Direccionalidad: cada requerimiento usa un URI.
- URI: `scheme://host:port/path?queryString`
 - `scheme`: puede no ser HTTP (FTP o HTTPS).
 - `host`: un nombre DNS o dirección IP.
 - `port`: un puerto específico o por default.
 - `path`: segmentos de texto delimitados por `"/`.
 - `queryString`: parámetros representados como pares `name=value`. Cada par está delimitado por `"&"`.

Principio: Interfaz uniforme (I)

- HTTP
 - GET - operación de SOLO lectura
 - idempotente (el mismo para cualquiera)
 - seguro (ningún cambio en el estado del servidor)
 - puede incluir parámetros en el URI

Idempotente: la ejecución repetida de una petición con los mismos parámetros sobre un mismo recurso tendrá el mismo efecto en el estado de nuestro recurso en el sistema si se ejecuta 1 o N veces.

Principio: Interfaz uniforme (III)

HTTP

POST - No es idempotente
- No es seguro

El consumidor de un API que no reciba una respuesta del servidor acerca del resultado del procesamiento de la petición (salta un timeout). En ese caso, ¿qué hacemos?, ¿repetimos nuevamente la petición?, ¿continuamos como si no hubiese pasado nada?.

Si no es idempotente, no podremos repetir la operación.

Elementos REST

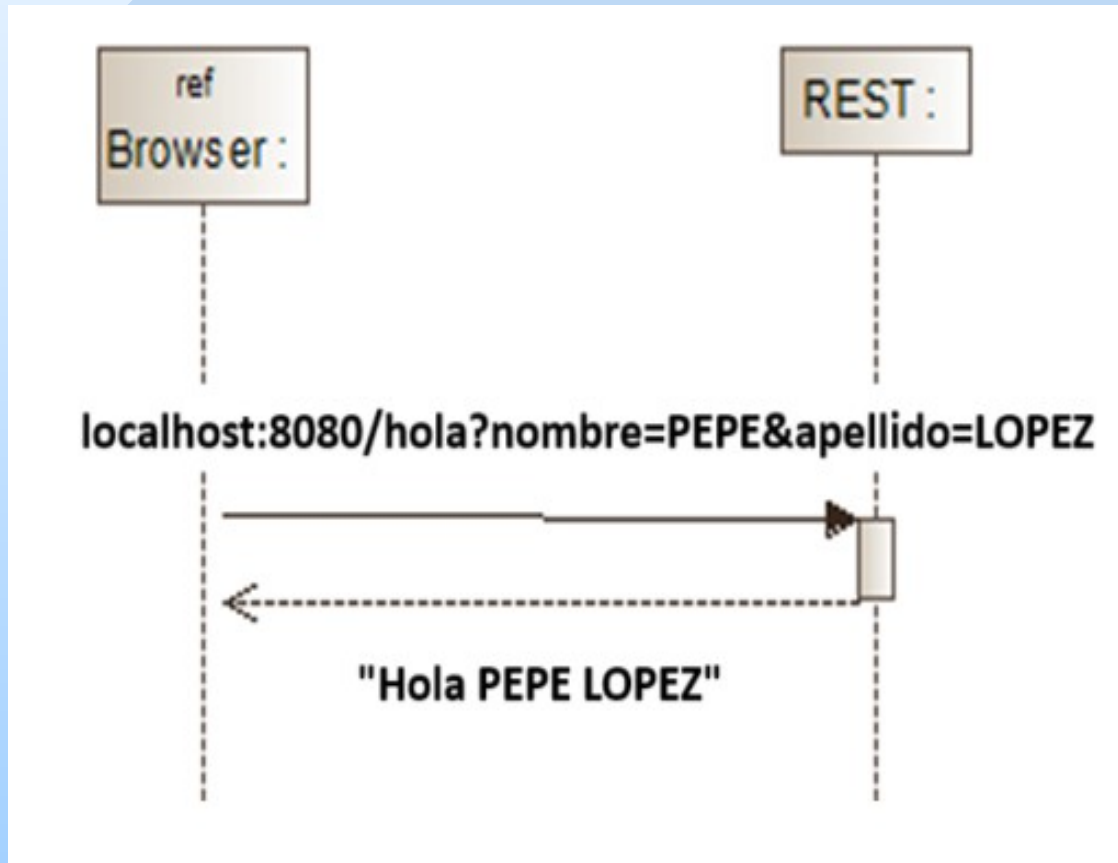
Interface uniforme (GET, PUT, POST, DELETE)

Orientado a los recursos

Simple

HTTP	Method	CRUD	Desc.
POST	CREATE	Create	-
GET	RETRIEVE	Retrieve	Seguro, Idempotente
PUT	UPDATE	Update	Idempotente
DELETE	DELETE	Delete	Idempotente

Saludar con Spring Boot con Parametros



Spring con Parametros

```
public class SpringBootHolaMundo {...
```

```
    @GetMapping("/hola2") // usado para el metodo get de HTTP
```

```
    public String holaEjemplo2(
```

```
        @RequestParam(value = "nombre",
```

```
                        defaultValue = "NombreDefault")
```

```
        String nbre,
```

```
        @RequestParam(value = "apellido",
```

```
                        defaultValue = "ApellidoDefault")
```

```
        String ape) {
```

```
        return "Hola " + nbre + " " + ape; }
```

Http://localhost:8080/hola2

http://localhost:8080/hola2?nombre=PEPE&apellido=LOPEZ

Ejemplo de REST

Saber ubicación de un número IP como país, región, ciudad, código postal, latitud, longitud, time-zone, moneda, ...

`http://api.ipstack.com/190.6.210.210?
access_key=ef241301df98d5d5cc52cdcdbd615c24`

Ver: <https://ipstack.com/documentation>

Ejemplos:

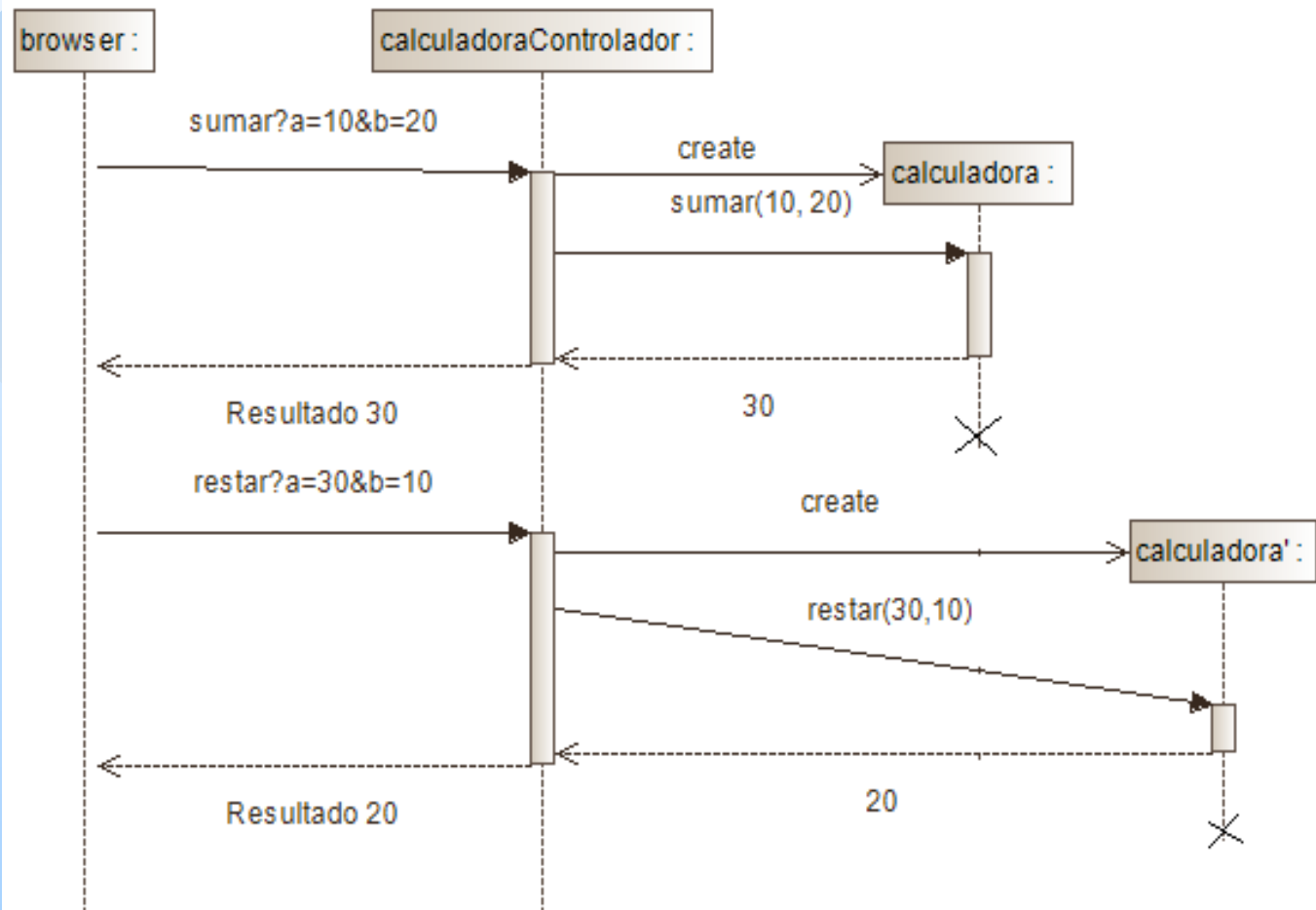
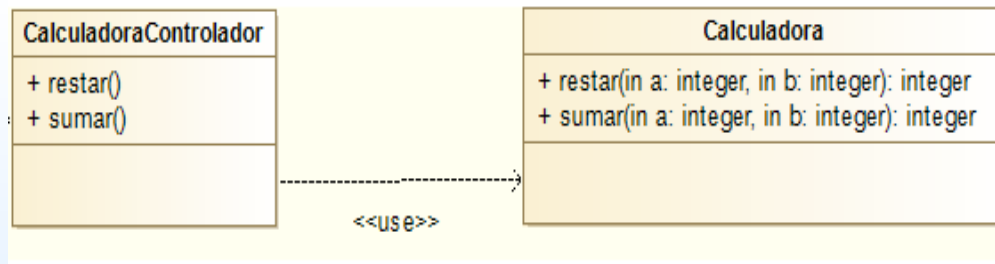
`http://api.ipstack.com/<IP>?access_key =
YOUR_ACCESS_KEY & output = json`

`http://api.ipstack.com/<IP>?access_key =
YOUR_ACCESS_KEY & output = xml`

Spring Boot Calculadora (REST)

SpringBootCalculadora

ES SIN ESTADO.



Spring Boot Calculadora: Main y Calculadora

@SpringBootApplication

```
public class CalculadoraMain {  
    public static void main(String[] args) throws Exception {  
        System.setProperty("server.port", "8081"); // cambiar el port  
        SpringApplication.run(CalculadoraMain.class, args);  
    }  
}
```

```
public class Calculadora {  
    public Integer sumar (Integer a, Integer b) {  
        return a + b;    }...}
```

Spring Boot Calculadora: Controlador

```
@RestController
```

```
@RequestMapping("/api") // agrega el camino .../api/sumar
```

```
public class CalculadoraControlador {
```

```
    @GetMapping("/sumar")
```

```
    public Integer sumar(@RequestParam(name = "a" ,  
                                defaultValue = "0") Integer a,  
                        @RequestParam(name = "b",  
                                defaultValue = "0") Integer b) {
```

```
        Calculadora c = new Calculadora();
```

```
        Integer resSuma = c.sumar(a, b);
```

```
        return resSuma;
```

```
    } // completar las operaciones faltantes
```

Microservicio con Go

package main

Import (

 "net/http"

 "github.com/gin-gonic/gin")

type album struct {

 ID string `json:"id"`

 Title string `json:"title"`

 Artist string `json:"artist"`

 Price float64 `json:"price"`

}

var albums = []album{

 {ID: "1", Title: "Blue Train", Artist: "John Coltrane", Price: 56.99},

 {ID: "2", Title: "Jeru", Artist: "Gerry Mulligan", Price: 17.99},

 {ID: "3", Title: "Sarah Vaughan and Clifford Brown", Artist: "Sarah Vaughan", Price: 39.99},

}

Microservicio con Go

```
func main() {  
    router := gin.Default() //Inicialice un enrutador Gin usando Default.  
    router.GET("/albums", getAlbums)  
    router.GET("/albums/:id", getAlbumByID)  
    router.POST("/albums", postAlbums)  
    router.Run("localhost:8080") //conectar enrutador al servidor e iniciarlo  
}  
  
func getAlbums(c *gin.Context) { c.IndentedJSON(http.StatusOK, albums)}  
func postAlbums(c *gin.Context) {...}  
func getAlbumByID(c *gin.Context) {  
    id := c.Param("id")  
    for _, a := range albums {  
        if a.ID == id {  
            c.IndentedJSON(http.StatusOK, a)  
            return    }    }  
    c.IndentedJSON(http.StatusNotFound, gin.H{"message": "album not found"}) }
```

Bibliografía

- Eberhard Wolff, Microservices Flexible Software Architecture, Addison-Wesley, 2017.
- Mark Richards, Software Architecture Patterns Understanding Common Architecture Patterns and When to Use Them, O'Reilly Media (2022). Capítulo 6.
- Bogunuva Mohanram Balachandar, RESTful Java Web Services, Third Edition, Packt Publishing (2017).